



YLang

COMPILED · EXPLICIT · FRIENDLY

The YLang Book

A beginner-friendly guide to YLang 1.0.0

How to install it, how to write programs, and how to fix your mistakes

Contents

Chapter 1. What is YLang?	2
Chapter 2. Installing YLang	3
Chapter 3. Your First Program	5
Chapter 4. The Four Commands	6
Chapter 5. Variables and Constants	7
Chapter 6. Types	9
Chapter 7. Printing and F-Strings	11
Chapter 8. Operators	12
Chapter 9. Making Decisions with if	13
Chapter 10. Repeating Things with loop	14
Chapter 11. Functions	15
Chapter 12. Reading Errors and Using fix	16
Chapter 13. Runtime Errors	17
Chapter 14. A Complete Program	18
Chapter 15. What YLang 1.0.0 Does Not Have	19
Chapter 16. Quick Reference	20

Every code sample in this book was compiled and run with the real YLang 1.0.0 compiler, so the outputs you see are the outputs you will get.

Chapter 1. What is YLang?

YLang is a small programming language. You write your program in a text file that ends with `.yl`. Then the YLang compiler checks your program and turns it into a real program your computer can run.

Here is what makes YLang special:

- **It is compiled.** There is no interpreter. YLang translates your code to C, then uses GCC or Clang to build a native executable.
- **Types are explicit.** You always say what kind of value a variable holds (`int`, `string`, and so on).
- **Errors are friendly.** Every error has a code (like `E2001`), the exact line and column, a picture of your code with an arrow, and a hint.
- **It is small on purpose.** Version 1.0.0 has a short list of features, and it does them carefully.

YLang is written in C and released under the MIT License.

Chapter 2. Installing YLang

YLang is distributed as source code. You build the compiler once on your machine, and then you can use it forever.

2.1 What you need

- **Linux.** Fedora and Ubuntu are the main tested systems. On Windows, the usual way is to use WSL (Windows Subsystem for Linux) and follow the Linux steps inside it. This is a suggestion from me, not something the YLang docs promise.
- **A C compiler:** `gcc` or `clang`. You need it twice: once to build YLang itself, and again every time YLang builds your programs.
- `make`, the build tool.

Install them on **Ubuntu or Debian**:

```
sudo apt install build-essential clang make
```

Install them on **Fedora**:

```
sudo dnf install gcc make clang
```

2.2 Get the source

You have the project as a zip file (`ylang-1_0_0-github-repo.zip`). Unzip it and go inside the folder:

```
unzip ylang-1_0_0-github-repo.zip -d ylang
cd ylang
```

If you cloned it from GitHub instead, just `cd` into the cloned folder.

2.3 Build the compiler

```
make
```

This creates the compiler at `build/ylang`.

2.4 Run the tests (recommended)

```
make test
```

If everything is fine, the last line says:

```
All YLang tests passed.
```

2.5 Check that it works

```
./build/ylang --version
```

Expected output:

```
YLang compiler 1.0.0 (C backend: GCC/Clang)
```

2.6 Install it system-wide (optional)

Right now you must type `./build/ylang` every time. If you want to type just `ylang` from any folder, install it:

```
sudo make install
```

This copies it to `/usr/local/bin/ylang`. To install in your home folder without `sudo`:

```
make install PREFIX="$HOME/.local"
```

(Make sure `$HOME/.local/bin` is in your `PATH`.) To remove it later:

```
sudo make uninstall
```

In the rest of this book, I write `ylang` for the command. If you did not install it, write `./build/ylang` instead.

Chapter 3. Your First Program

Create a file called `hello.yl`:

```
function main() -> int {  
    print("Hello, YLang!");  
    return 0;  
}
```

Now build it and run it:

```
ylang build hello.yl -o hello  
./hello
```

Output:

```
Hello, YLang!
```

Let us read the program line by line:

- `function main() -> int {` starts a function named `main`. Every YLang program must have one. It is where the program starts. `-> int` means it gives back an integer when it finishes.
- `print("Hello, YLang!");` shows text on the screen.
- `return 0;` ends the program. `0` means success.
- `}` closes the function.

Two rules you will see everywhere:

- 1 Every statement ends with a semicolon ;.**
- 2 Blocks of code live inside curly braces { }.**

Comments start with `//` and go to the end of the line:

```
// This is a comment. YLang ignores it.  
print("Hi"); // This is also a comment.
```

The main function

`main` can have two forms. Both take no parameters:

```
function main() -> int { return 0; }
```

```
function main() -> void { }
```

`void` means "returns nothing".

Chapter 4. The Four Commands

The `ylang` tool has four commands you will use.

Command	What it does
<code>ylang check file.yl</code>	Checks your program for mistakes. Does not make an executable. Fast.
<code>ylang build file.yl -o name</code>	Checks your program, then makes an executable called <code>name</code> .
<code>ylang emit-c file.yl -o out.c</code>	Saves the generated C code so you can read it.
<code>ylang fix file.yl -o fixed.yl</code>	Fixes one specific typo automatically (see Chapter 12).

Other useful flags:

- `ylang --help` shows help.
- `ylang --version` shows the version.
- `--cc clang` (or `--cc gcc`) chooses which C compiler `build` uses:

```
ylang build hello.yl -o hello --cc clang
```

If your program has errors, `build` will **not** create an executable. That is a feature: you never run a half-checked program.

Chapter 5. Variables and Constants

A **variable** is a named box that holds a value. In YLang you create one with `let`, then the **type**, then the **name**:

```
let int score = 100;
let string name = "YLang";
let bool enabled = true;
```

You can also create a variable without a value and give it one later:

```
let int age;
age = 16;
```

To change a variable, use `=`:

```
score = score + 10;
```

Constants

A **constant** is a variable that can never change. Add `const` after `let`:

```
let const int LIMIT = 100;
```

A constant must be given a value right away. If you try to change it, YLang stops you:

```
function main() -> int {
    let const int MAX = 100;
    MAX = 200;
    return 0;
}
```

```
error[E2022]: Cannot assign to a const variable.
--> tests/const-assignment.yl:3:9
  |
3 |     MAX = 200;
  |         ^
= help: Keep 'const' and remove this assignment, or remove
'const' if the variable should be mutable.
```

The "initialized before reading" rule

YLang will not let you read a local variable unless it is sure the variable has a value on every possible path:

```
function main() -> int {
    let int score;
    if (true) {
        score = 10;
    }
    print(score);
    return 0;
}
```

```
error[E2021]: This variable may be used before it is initialized.
--> examples/uninitialized.yl:6:11
    |
  6 |     print(score);
    |           ^^^^^
    = help: Assign a value to 'score' before reading it.
```

The fix is to give `score` a value in an `else` branch too, or to give it a starting value when you declare it. This version is accepted, because both paths assign `score`:

```
let int score;
if (true) {
    score = 10;
} else {
    score = 20;
}
print(score);
```

Global variables

Variables declared outside any function are **global**. A global variable without a value gets a safe default: `0` for numbers, `""` for strings, `false` for booleans. Still, it is better to write the value yourself.

```
let string blank;
let int counter;

function main() -> int {
    print(f"blank='{blank}', counter={counter}");
    return 0;
}
```

```
blank='', counter=0
```

Scope and shadowing

A variable lives inside the `{ }` block where you created it. You may reuse a name in an inner block (this is called *shadowing*), but you cannot declare the same name twice in the same block.

Chapter 6. Types

YLang has these types:

Type	What it holds	Example
<code>int</code>	Whole numbers (signed 64-bit)	<code>42</code> , <code>-7</code>
<code>float</code>	Decimal numbers (64-bit)	<code>1.25</code>
<code>bool</code>	<code>true</code> or <code>false</code>	<code>true</code>
<code>char</code>	One single byte (a letter like <code>'Y'</code>)	<code>'Y'</code>
<code>string</code>	Text. It cannot be changed after creation.	<code>"Hello"</code>
<code>void</code>	"Nothing". Only for function return types.	(none)

```
let int n = 42;
let float ratio = 1.25;
let bool ready = true;
let char initial = 'Y';
let string message = "Hello";
```

No automatic conversions

YLang never mixes types behind your back. You cannot add an `int` and a `float`:

```
let int a = 5;
let float b = 2.0;
print(a + b);
```

```
error[E2034]: Arithmetic operands must have the same numeric type.
--> file.yl:4:13
  |
4 |     print(a + b);
  |           ^
  = help: Use two int values or two float values; convert
         explicitly when needed.
```

Similarly, you cannot put text into an `int` variable:

```
let int age = "sixteen";
```

```
error[E2001]: Variable initializer type mismatch.
...
  = help: 'age' has type int, but its initializer has type string.
```

Other type rules

- `%` (remainder) works only on `int`.

- An `if` condition must be a `bool`. You cannot write `if (1)`.
- Strings can be compared with `==` and `!=`.
- A `char` is one byte, so it is meant for plain ASCII letters, not for emoji or Arabic letters.

Chapter 7. Printing and F-Strings

print

`print` shows values and then starts a new line. If you give it several values, it separates them with one space:

```
print("answer:", 42);  
print("odd", 7, 1.5, true, 'x');
```

```
answer: 42  
odd 7 1.5 true x
```

You can print `int`, `float`, `bool`, `char`, and `string` values.

`print` is a **statement**, not a function. That means you write `print(x);` on its own line. You cannot use it inside another expression.

F-strings

Put an `f` before the opening quote and write expressions inside `{ }`:

```
let const string project = "YLang";  
let int sum = 30;  
  
print(f"Welcome to {project}!");  
print(f"next={sum + 1}");
```

```
Welcome to YLang!  
next=31
```

You can use variables and operators inside the braces. **You cannot call functions or assign values inside the braces.** Do the work on the line before:

```
let string kind = describe(i);    // call the function first  
print(f"{i} is {kind}");         // then use the result
```

To write a literal brace, double it:

```
print(f"Progress: 100% {{complete}}");
```

```
Progress: 100% {complete}
```

Escape characters

Inside a normal string, `\n` means a new line, and `\"` means a quote character.

Chapter 8. Operators

Kind	Operators
Arithmetic	+ - * / %
Comparison	== != < <= > >=
Boolean	and or not
Negative sign	- (example: -5)
Grouping	()

Note that boolean operators are **words**, not symbols: write `and`, `or`, `not`, never `&&`, `||`, `!`.

```
let bool ok = (5 > 3) and not (2 > 4);
```

Order of operations

From strongest to weakest: unary (`-`, `not`), then `* / %`, then `+ -`, then comparisons, then `and`, then `or`. When in doubt, use parentheses.

Assignment is a statement

`x = 5;` is a full statement. You cannot hide an assignment inside another expression.

Chapter 9. Making Decisions with if

```
function main() -> int {
  let int level = 12;
  if (level < 10) {
    print("beginner");
  } else if (level < 20) {
    print("intermediate");
  } else if (level < 30) {
    print("advanced");
  } else {
    print("expert");
  }
  return 0;
}
```

intermediate

Rules:

- The condition goes in parentheses ().
- The body always goes in curly braces { }, even for one line.
- The condition must be a `bool`.
- `else if` and `else` are optional.

Chapter 10. Repeating Things with loop

YLang 1.0.0 has **one** kind of loop: `loop()`. It repeats forever, until you stop it with `break`.

```
function main() -> int {
    let int count = 0;
    loop() {
        count = count + 1;
        print(f"Count: {count}");
        if (count >= 3) {
            break;
        }
    }
    return 0;
}
```

```
Count: 1
Count: 2
Count: 3
```

- `break`; leaves the loop immediately.
- `continue`; jumps back to the start of the loop and skips the rest of the current round.
- Both only work inside a loop.

There is no `for` and no `while`. If you need them, build them with `loop()` plus an `if` and `break`. For example, a "while count is less than 5" loop looks like this:

```
loop() {
    if (not (count < 5)) {
        break;
    }
    count = count + 1;
}
```

Chapter 11. Functions

A function is a named, reusable piece of code. You write the parameter types and the return type:

```
function add(int a, int b) -> int {  
    return a + b;  
}  
  
function main() -> int {  
    print("Sum:", add(10, 20));  
    return 0;  
}
```

Sum: 30

The shape is:

```
function name(type param, type param) -> returnType {  
    ...  
}
```

Points to remember:

- Use `return value;` to send a value back.
- A function that returns `void` returns nothing and does not need `return`.
- A function that returns a value must return on every path.
- You cannot have two functions with the same name (no overloading).
- A function that returns `void` is still called as a normal statement:

```
function greet(string who) -> void {  
    print(f"Hi {who}");  
}  
  
function main() -> void {  
    greet("Sam");  
}
```

Hi Sam

Chapter 12. Reading Errors and Using fix

A YLang error looks like this:

```
error[E1002]: Expected ';' after variable declaration.
--> tests/missing-semicolon.yl:3:5
  |
3 |     print(age);
  |     ^^^^^
= help: The declaration may be missing ';' before this token.
```

How to read it:

- 1 **E1002** is the error code. Codes starting with **E1** are syntax (grammar) errors. Codes starting with **E2** are meaning errors such as wrong types or unknown names.
- 2 **tests/missing-semicolon.yl:3:5** means file, line 3, column 5.
- 3 The line of code is shown, and **^** points at the problem.
- 4 **help** gives a hint.

A useful tip: the arrow shows where YLang **noticed** the problem. In the example above, the missing **;** is actually at the end of the **previous** line (**let int age = 16**), and YLang noticed it when it reached **print**. So if the arrow looks wrong, check the line above.

At the end, YLang tells you the result:

```
ylang: build failed with 1 error(s); no executable was produced.
```

The fix command

There is one typo YLang can fix for you: writing **pritrn(...)** instead of **print(...)**.

```
ylang fix broken.yl -o fixed.yl
ylang check fixed.yl
```

fix never changes your original file. It writes a new file, and you must choose the name with **-o**. It fixes only this one typo and nothing else. Always run **check** on the new file afterward. For example, fixing **examples/type-error.yl** repairs the **pritrn**, but the type error on the line above it is still there, because **fix** does not guess about that.

Chapter 13. Runtime Errors

Some mistakes only appear while the program is running. YLang adds checks for two of them. When one happens, the program prints a message and stops with exit status **70**.

Division by zero:

```
function main() -> int {  
  let int x = 4 / 0;  
  print(x);  
  return 0;  
}
```

YLang runtime error: integer division by zero

Integer overflow. An `int` holds numbers up to 9223372036854775807. Going past that is caught:

```
function main() -> int {  
  let int max = 9223372036854775807;  
  let int overflow = max + 1;  
  print(overflow);  
  return 0;  
}
```

YLang runtime error: integer overflow in addition

The smallest `int` is fine to use: `-9223372036854775808` works.

Chapter 14. A Complete Program

Here is a program that uses almost everything from this book. Save it as `numbers.yl`:

```
// Counts from 1 to 10 and describes each number.
let const int LIMIT = 10;

function is_even(int n) -> bool {
    return n % 2 == 0;
}

function describe(int n) -> string {
    if (is_even(n)) {
        return "even";
    }
    return "odd";
}

function main() -> int {
    let int i = 0;
    let int total = 0;

    loop() {
        i = i + 1;
        if (i > LIMIT) {
            break;
        }
        total = total + i;
        let string kind = describe(i);
        print(f"{i} is {kind}");
    }

    print(f"Total: {total}");
    if (total == 55 and not (total < 0)) {
        print("All good!");
    }
    return 0;
}
```

Build and run:

```
ylang check numbers.yl
ylang build numbers.yl -o numbers
./numbers
```

The last lines of the output:

```
9 is odd
10 is even
Total: 55
All good!
```

Chapter 15. What YLang 1.0.0 Does Not Have

YLang is honest about its limits. These are **not** available in 1.0.0, and programs that use them are rejected:

- Arrays and lists
- Pointers and references
- Classes and objects
- `try / catch`
- `for` and `while` loops (use `loop()`)
- Modules and imports (each file is one whole program)
- Generics

Be careful with the rejection: when you use something unsupported such as `while`, YLang does not say "while is not supported". It reports an ordinary syntax error (for example, `Expected ';' after expression`). If you get a strange syntax error on a line that looks right, ask yourself whether you used a feature from this list.

A few more things to know:

- `char` is one byte, not a Unicode letter.
- Some f-string results stay in memory until the program ends. A program that creates millions of them in a loop may use a lot of memory.
- YLang programs are normal native programs with your user's permissions. The YLang authors say not to use this release for security-critical code.

Chapter 16. Quick Reference

```
// comment
let int x = 1;           // variable
let const int Y = 2;     // constant
let string s = "hi";    // string
let float f = 2.5;      // float
let bool b = true;      // bool
let char c = 'A';       // char

x = x + 1;               // assignment
print("x is", x);        // print
print(f"x is {x}");      // f-string

if (x > 1 and b) { }     // condition
else if (not b) { }
else { }

loop() {                 // loop
    if (x > 5) { break; }
    continue;
}

function add(int a, int b) -> int { // function
    return a + b;
}
```

I want to...	Command
Build the compiler	make
Test the compiler	make test
Check my program	ylang check file.yl
Build my program	ylang build file.yl -o name
See the generated C	ylang emit-c file.yl -o out.c
Fix pritrn typos	ylang fix file.yl -o fixed.yl

Happy coding with YLang!